Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Kotlin Корутины

- suspend function
- scope
- builders
- exceptions
- flow

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011804991924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 32. Анимация. AnimateValueAsState

28 сентября 2023

В этом уроке работаем с функцией AnimateValueAsState

Мы добрались до верхнеуровневых функций анимации. Они позволяют писать меньше кода. И хотя при этом мы теряем часть функционала, чаще всего этот функционал не нужен и можно прожить без него.

Начнем с функции animateValueAsState. Чтобы не возиться с конвертерами, существуют версии этой функции для разных типов: Int, Float, и т.п.



На примере одной из них мы рассмотрим, что это за функции и как они работают.

Пример будет простым: Checkbox и Text, отображающий число. По клику на чекбокс запускаем анимацию, которая плавно меняет число в Text от 100 до 500 и обратно.

Этот пример мы сначала сделаем с Animatable. А потом перепишем его на animateIntAsState, чтобы увидеть разницу.

Реализация Animatable:

```
@Composable
fun HomeScreen() {
    Column {
        val scope = rememberCoroutineScope()
        val animatableCount = remember {
            Animatable(initialValue = 100, typeConverter = Int.VectorConverter)
        }
        var checked by remember { mutableStateOf(false) }

        Checkbox(checked = checked, onCheckedChange = {
            checked = it
            scope.launch {
                animatableCount.animateTo(
                    targetValue = if (checked) 500 else 100,
                    animationSpec = tween(1000)
                )
            }
        })

        Text(text = "${animatableCount.value}", fontSize = 30.sp)
    }
}
```

Получаем scope, создаем Animatable и по клику на чекбокс стартуем анимацию до 500 или до 100.

Результат



Этот код можно сделать проще с помощью функции animateIntAsState:

```
@Composable
fun HomeScreen() {
    Column {
        var checked by remember { mutableStateOf(false) }
        Checkbox(checked = checked, onCheckedChange = {checked = it })

        val animatedCount = animateIntAsState(
            targetValue = if (checked) 500 else 100,
            animationSpec = tween(1000),
            label = "Count"
        )
        Text(text = "${animatedCount.value}", fontSize = 30.sp)
    }
}
```

У нас больше нет Animatable и scope. И по клику на чекбокс мы ничего не запускаем.

Триггером для старта анимации теперь является рекомпозиция. Когда мы кликаем на чекбокс, меняется значение State checked. Это приводит к тому, что перезапускается HomeScreen, т.к. он читает значение checked. Перезапуск HomeScreen приводит к повторному запуску функции animateIntAsState. И там уже происходит вызов анимации до нового значения targetValue.

Под капотом animateValueAsState происходит примерно следующее:

```
@Composable
fun <T, V : AnimationVector> animateValueAsState(
    targetValue: T,
    ...
): State<T> {
    val animatable = remember { Animatable(targetValue, ...) }

    val channel = remember { Channel<T>(Channel.CONFLATED) }
    SideEffect {
        channel.trySend(targetValue)
    }
    LaunchedEffect(channel) {
        for (target in channel) {
            val newTarget = channel.tryReceive().getOrNull() ?: target
            launch {
                if (newTarget != animatable.targetValue) {
                    animatable.animateTo(newTarget, ...)
                }
            }
        }
    }

    return animatable.asState()
}
```

При первом запуске создается объект Animatable. Причем значение targetValue используется, как начальное значение (initialValue) для Animatable.

Также только при первом запуске создается котлиновский Channel. В этот канал передается targetValue при каждом перезапуске функции.

В LaunchedEffect крутится бесконечный цикл, который ждет новое значение targetValue из канала. Как только оно приходит, выполняется проверка, что оно отличается от текущей цели анимации и запускается анимация к этому значению.

В итоге каждый перезапуск функции animateValueAsState приводит к запуску анимации, если значение targetValue поменялось.

В качестве еще одного примера можно рассмотреть более интересный случай, когда target значение приходит из ViewModel.

Например, мы делаем приложение для отображения статистики веб-сайта и хотим отображать текущее число посетителей. В ViewModel мы каждые пару секунд получаем новые данные и помещаем их в Flow, который используется в UI.

Код экрана:

```
@Composable
fun HomeScreen(
    homeViewModel: HomeViewModel = hiltViewModel()
) {
    val animatedCount = animateIntAsState(
        targetValue = homeViewModel.visitorCount.collectAsState().value,
        animationSpec = tween(1000),
        label = "Count"
    )
    Text(
        text = "Visitors: ${animatedCount.value}",
        fontSize = 30.sp
    )
}
```

Функция animateIntAsState позволяет нам не просто обновлять значение счетчика в Text, а делать это с анимацией.

Результат:



Код HomeViewModel, в котором мы генерируем случайное число посетителей каждые 2 секунды:

```
@HiltViewModel
class HomeViewModel @Inject constructor() : ViewModel() {

    val visitorCount = MutableStateFlow(0)

    init {
        viewModelScope.launch {
            while (isActive) {
                delay(2000)
                visitorCount.value = Random.nextInt(100, 500)
            }
        }
    }
}
```

Параметры

Кроме target значения и спецификации у функции animateValueAsState есть параметры label и finishedListener.

Значение label используется для работы с анимацией в Animation Preview в Android Studio. Об этом мы еще подробно поговорим в уроке про инструменты студии для Compose.

A finishedListener - слушатель, который сообщит, что анимация завершилась и предоставит нам target значение.

Animatable VS animateValueAsState

Возможно, возникает вопрос: в чем разница между Animatable и animateValueAsState. Что и когда использовать?

Используйте Animatable, если вам нужны его возможности, которые мы рассматривали в Уроке 30: isRunning, velocity, lowerBound\upperBound, stop() и тп.

Но чаще всего нам нужна просто анимация. В этом случае можно использовать функцию animateValueAsState, что даст более простой и лаконичный код.

Еще одно отличие в том, что animateValueAsState можно вызвать только в Compose коде. А с Animatable можно работать и за пределами Compose.

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.